

ECE 3401 Digital Systems Design – Spring 2026

Programming Assignment 4: Digital Design of a Pipelined RISC-V with GEMM Accelerator

Due April 30, 2026 @ 11:59 PM on HuskyCT

So far, we've used Latency to speed up the matrix multiplications. However, simply decreasing the latency does not guarantee better performance beyond a certain threshold. To understand why, we can turn to **Little's Law**, which provides a useful framework for reasoning about the relationship between throughput, latency, and parallelism. Little's Law is a fundamental result from queueing theory that states that:

$$\text{Required Parallelism} = \text{Latency} \times \text{Throughput}$$

Little's Law captures an important relationship: to maintain a steady rate of completing work (throughput) when each piece of work takes time to finish (latency), you need to have multiple pieces of work in progress simultaneously (parallelism). Intuitively, if operations take time to complete, you need more of them happening in parallel to keep results flowing out at a constant rate.

In PA3, either RISC-V processor was allowed to perform its operations, or the GEMM accelerator was active to compute the dot products. This implies that only one GEMM instance is performed at a given time. Due to this sequential processing the throughput of processing multiple GEMM instance is $\text{Throughput} = \frac{1}{\text{Latency}}$. Therefore, the achieved parallelism for PA3 design is 1. This makes PA3 non-pipelined design, since our achieved parallelism is 1. This confirms that the hardware is only ever working on a single matrix multiplication at a time, leaving no overlap between loading in RISC-V, computing in GEMM, and storing in RISC-V. The only speedups in PA3 originate from the acceleration of GEMM operations in the accelerator, reducing the latency component.

The main goal of this assignment is to improve the achieved parallelism or the average number of operations (RISC-V and GEMM) in-flight. This will be done by extending the RISC-V with GEMM accelerator to support pipelined execution of RISC-V instructions with the GEMM dot product computations. The design will use **double-buffering** to allow RISC-V to load the *next gemm's* data into the register file, while the GEMM accelerator computes dot products using the *current gemm's* data from previously loaded data in the register file. The overlap of RISC-V and GEMM operations unlocks parallelism, leading to throughput improvements for the PA4 design.

To support double-buffering, the RISC-V ISA is extended to support two separate `gemm` instructions. The `Gemm_start_1` instruction uses RISC-V registers x5-x8 (ABI names t0-t2, s0, see [riscv-uconn.pdf](#)) for matrix A, x9-x12 (s1, a0-a2, see [riscv-uconn.pdf](#)) for matrix B, and x13-x16 (ABI names a3-a6, see [riscv-uconn.pdf](#)) for matrix C. On the other hand, `Gemm_start_2` instruction uses RISC-V registers x17-x20 (ABI names a7, s2-s4, see [riscv-uconn.pdf](#)) for matrix A, x21-x24 (s5-s8, see [riscv-uconn.pdf](#)) for matrix B, and x25-x28 (ABI names s9-s11, t3, see [riscv-uconn.pdf](#)) for matrix C. A ***.asm** program interleaves the RISC-V load (LW) and store (SW) instructions with the `gemm1` and `gemm2` instructions to achieve pipelining of operations between the RISC-V processor and the GEMM accelerator. Several **gemm*.asm** programs are provided that perform various overlapping and/or non-overlapping operation sequences. Like PA3, you must assemble these programs and run the **make <test_name>** at the command line.

logic must follow the IDLE and MAC states to control the FSM, like PA1. At any given time, the FSM handles one of the two GEMM operations, so the appropriate `gemm_active_1` or `gemm_active_2` output must be asserted to indicate which GEMM instance is actively performing the MAC operations. These signals identify that the GEMM unit is using the register file's x5-x16 registers for `Gemm_start_1` and x17-x28 registers for `Gemm_start_2`. To support pipelined execution, independent register file ports enable decoupled execution of these GEMM operations, while the RISC-V processor can perform its instructions in parallel. The datapath process in the GEMM module updates the register file on the rising edge of the clock. The synchronous clock logic is already implemented in `Register_file.vhd`, so the GEMM module's datapath is combinational logic. The index calculation for `A_addr`, `B_addr`, and `C_addr` outputs must be computed each cycle based on the `i`, `j`, and `k` signals from the control FSM. These need to be computed separately for `Gemm_start_1` and `Gemm_start_2` with appropriate logic for selection during active GEMM execution. Moreover, the 64-bit `mul_res` and 32-bit `acc_res` signals are provided for the MAC datapath logic computations to determine the 32-bit `gemm_output`.

GEMM-RV5 Implementation

The `GEMM_RV5.vhd` is primarily responsible for the interface logic needed for the RISC-V processor and GEMM module to orchestrate logic via the shared Register file with independent read and write ports. At the high level, when GEMM module is active, the RISC-V processor is allowed to proceed with execution of normal instruction processing, and halts when it performs a load (LW) or store (SW) operation using the registers that are concurrently being used by the active GEMM module.

When GEMM is active, either `gemm_active_1` or `gemm_active_2` is asserted and the `A_addr`, `B_addr`, and `C_addr` signals indicate the operands for reading the three input and the output matrix data from the register file. The `gemm_output` is written to the register file at the `C_addr`. The GEMM_RV5 logic must map the GEMM to the appropriate the register file ports. The 32-bit `instruction` signal from the `Imem_Dmem.vhd` is used to compute the addresses for the register file read and write ports dedicated for RISC-V processor. Furthermore, the `RegWrite` signal from the `Controller.vhd` and the 32-bit `regf_wd` signal from the `Datapath.vhd` provides the needed write enable and data for the RISC-V processor. The GEMM_RV5 logic then computes the register file read and write port addresses, the write enable and the write data logic for the RISC-V processor.

During GEMM active execution, the RISC-V processor may need to halt instruction processing. This is accomplished by de-asserting the `pc_en` signal to `Datapath.vhd` to freeze the PC logic. Moreover, when RISC-V processor is halted, all architectural state updates need to be disabled. The `dmem_we` signal to the `Imem_Dmem.vhd` must be de-asserted to ensure no memory updates are allowed when the RISC-V processor is halted. The RISC-V processor halts when `gemm_active_1` is active and RISC-V is executing a load (LW) or store (SW) instruction using the x5-x16 registers, or `gemm_active_2` is active, and RISC-V is executing a load (LW) or store (SW) instruction using the x17-x28 registers. This implies that RISC-V and GEMM overlap cannot occur when both modules may end up reading and/or writing to the same registers.

Design Verification

We have provided a fully functional `testbench.vhd`. After initialization and reset, a program test being run is executed by the design. The testbench waits for the `terminate` signal and waits for 50ps before reporting the completion of the simulation. If a test does not complete in 500ns, the testbench reports and ERROR timeout message. This process can be performed sequentially for each of the **gemm*.asm** programs in the `assembled_tests` directory.

Deliverables

Please submit the following:

1. Your modified Controller.vhd, GEMM.vhd and GEMM_RV5.vhd VHDL files.
2. Submit a report PDF with block diagrams for the GEMM and GEMM_RV5 modules clearly showing the input and output signals and behavioral components such as registers, multiplexors, arithmetic modules etc. Provide a short description of your design to get credit.
3. Submit a single PDF with a screenshot of the output waveforms for all program tests in the `assembled_tests` directory. You must use the `pa4.gtkw` to generate these waveforms.

In case your design is not fully functional, you will need to schedule a code review with the TA.